TRACE: Topology-Aware Visual Analytics for Tree Evolution

Category: Research

Paper Type: system

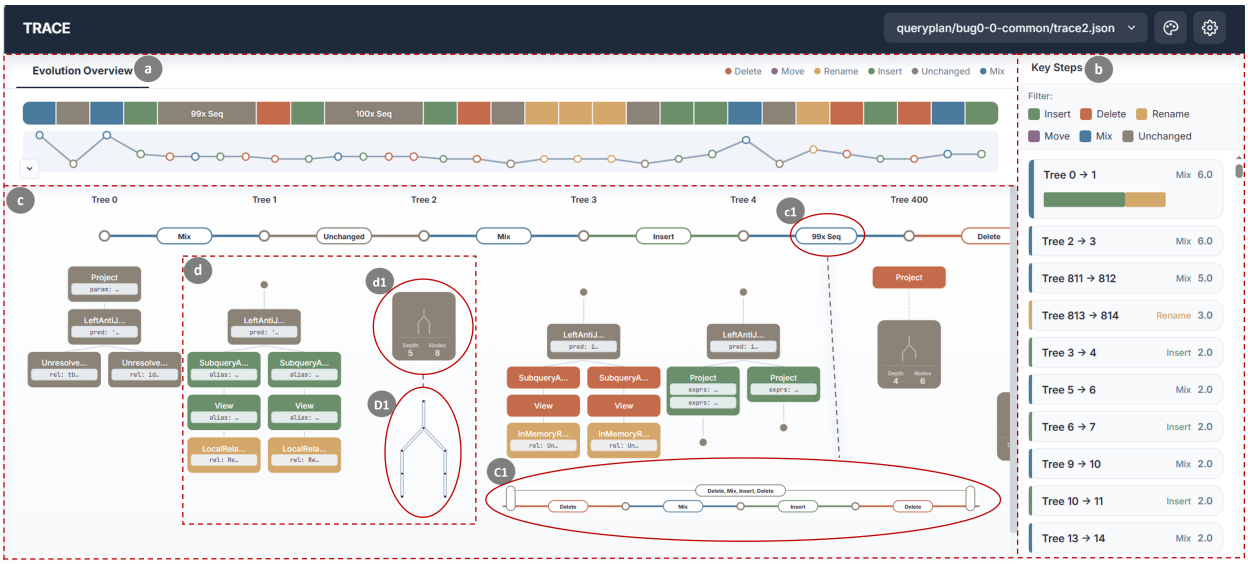

Fig. 1: The user interface of TRACE consists of multiple complementary views that support progressive exploration from global temporal patterns to local structural details. (a) Statistics View provides a global overview of structural change distribution over the entire evolution sequence, helping analysts identify salient intervals and potential transition points. (b) Hierarchy View presents the selected evolution region in a compact hierarchical form, supporting multi-granularity inspection of structural changes. (c) Key Steps View highlights critical transition steps within the selected interval. (d) Tree Views preserve the finest-grained tree structures and expose node-level edits for detailed inspection, enabling analysts to examine concrete edit operations within their structural context.

Abstract—Tree-structured data are common in many domains, such as organizational hierarchies, logical query plans in relational databases, and abstract syntax trees in program analysis. In these applications, tree structures often evolve over time. Analyzing tree evolution is important for understanding how tree structures change, identifying critical transition steps, and diagnosing abnormal or inefficient processes. However, visual analytics of tree evolution remains challenging. First, comparing tree topologies is difficult for analysts, and this challenge becomes greater in evolution analysis, where they must not only identify structural differences but also understand how one tree transforms into the next. Second, local topological changes are often scattered across large unchanged contexts, making it difficult to highlight critical differences without losing structural context. Third, critical changes in long evolution sequences are usually sparse, making it hard to summarize major trends while preserving key transition steps.

To address these challenges, we propose TRACE, a topology-aware visual analytics framework for the evolution of tree structures. TRACE first computes topological differences between adjacent trees and identifies changed nodes, representing the evolution as a sequence of local structural changes. It then extracts and organizes difference-aware substructures that preserve essential context while reducing visual complexity, enabling effective comparison of critical topological differences. Furthermore, TRACE traces the lifecycle of nodes throughout the evolution process, supporting localized temporal analysis of changes. Building on these localized analyses, TRACE simplifies long sequences based on the distribution of differences to reveal salient intervals, key transition steps, and overall evolutionary trends. Moreover, TRACE supports multi-granularity analysis from local changes to global evolution patterns.

We evaluate TRACE through case studies on multiple datasets and user interviews. The results show that TRACE helps analysts better understand evolution processes in tree-structured data, locate key structural changes, and identify important patterns and transitions in long sequences.

Index Terms—Tree Compraison, Tree Evolution, Multi-Granularity Analysis

1 Introduction

Tree-structured data are widely used to represent hierarchical information and model complex processes, such as organizational hierarchies, query plan trees in relational databases, reasoning trees in search and inference tasks, and abstract syntax trees (ASTs) in program analysis. In many real-world scenarios these trees are not static. They evolve over time or across steps as rules are applied, states are updated, search progresses, or human edits are introduced, forming sequences of tree transformations. Analyzing tree evolution is important for understanding how hierarchical structures change, identifying critical transition steps, and diagnosing abnormal or inefficient processes.

Compared with tree sequence analysis that treats trees primarily as a series of snapshots, Tree Evolution analysis focuses not only on what a tree looks like at a given step, but also with how one tree transforms into the next and why these changes occur. In many applications, the final tree only reflects the result of an evolving process, while obscuring key rewrites, backtracking behaviors, or local irregularities along the way. For example, in query optimization, an initial logical plan may go through a series of rule-based rewrites and structural adjustments before becoming an executable plan. Examining this process can help analysts understand optimizer behavior and locate the steps that lead to

incorrect results or performance degradation. Similarly, in reasoning and search tasks, tree expansion, pruning, and rollback reveal the exploration strategy of the system; in program evolution analysis, local modifications on ASTs and their propagated effects can help expose refactoring behavior and error sources. These examples suggest that trees should often be understood as evolving structures rather than isolated snapshots.

Despite its importance, visual analytics of Tree Evolution remains challenging in at least three aspects. First, prior studies have shown that even comparing relatively simple graphs can be difficult for analysts [], because topological comparison often disrupts cognitive continuity. This challenge becomes even greater in Tree Evolution analysis, where analysts must not only identify structural differences but also understand how one tree transforms into the next. Second, local topological changes are often scattered across large unchanged contexts, making it difficult to highlight critical differences without losing the structural context needed for interpretation. Third, critical changes in long evolution sequences are often sparse, making it hard to summarize major trends while preserving key transition steps.

Existing studies have explored related problems from several perspectives, such as domain-specific evolution analysis, dynamic tree visualization, structural comparision, and temporal visual analytics. Some approaches are designed for specific application domains. For example, QOVIS [] supports the analysis of query optimization process by leveraging domain knowledge. While such systems are effective in their target settings like the optimization rules. They rely on strong domain assumptions and are difficult to generalize to other tasks. Other approaches use animation [] or dynamic tree views [] to present transitions between adjacent steps, which can help reveal local continuity but become less effective for reviewing long sequences, locating critical steps, or comparing sparse changes across time. Temporal visual analytics methods can summarize trends and high-level stage-wise patterns, but they often ignore the topological structure and fine-grained change semantics of the trees.

To address this gap, we propose TRACE, a topology-aware visual analytics framework for the evolution of tree structures. TRACE first computes topological differences between adjacent trees and identifies changed nodes, representing the evolution as a sequence of local structural changes rather than a set of disconnected snapshots. It then extracts and organizes compact difference-aware substructures that preserve essential context while reducing visual complexity, enabling effective comparison of critical topological differences. Furthermore, TRACE traces the lifecycle of nodes throughout the evolution process to track the development of important information over time. Building on these local analyses, TRACE simplifies long sequences based on the distribution of differences to reveal salient intervals, key transition steps, and overall evolutionary trends. In this way, Building on these techniques, TRACE supports multi-granularity analysis from local changes to global evolution patterns.

The contributions of this work are as follows:

- We formulate design requirements of TRACE, which addresses the core challenges of Tree Evolution as a visual analytics problem, including topology-aware comparison, context-preserving difference analysis, and long-sequence summarization.
- We propose a configurable differencing method for adjacent tree structures to estimate corresponding changes and evolution processes, and further develop a compact difference-aware substructure extraction strategy that balances fidelity and cognitive load.
- We design and implement TRACE, a multi-granularity visual analytics framework that supports local topology comparison, cross-step evolution tracing, and long-sequence summarization of key processes.
- We demonstrate the effectiveness and practical value of TRACE through multiple case studies and user interviews in tasks involving tree evolution understanding, key change localization, and long-sequence analysis.

The remaining parts of the paper are organized as follows:

2 Related Work

In this section, we review related work from four aspects: methods for visualizing dynamic hierarchies, techniques for computing tree differences and modeling structural changes, approaches for summarizing long temporal sequences, and domain-specific systems that analyze evolving tree-like structures.

2.1 Dynamic Tree Visualization

Much existing research has studied how to visualize tree structure data evolve over time. One representative direction focuses on stable space-filling layouts for evolving hierarchies. Temporal Treemaps transforms a sequence of trees with known correspondences into a static nested treemap, enabling users to inspect appearance, disappearance, merge, and split events without relying solely on animation [12]. EvoCells proposes a treemap layout strategy tailored to evolving tree data, explicitly mapping tree updates to rectangle updates to improve layout stability under hierarchy changes [17]. Stable Voronoi treemap approaches further reduce unnecessary layout variation when hierarchies evolve, improving cross-time comparability while preserving the overall spatial organization [10].

Other methods explore static or semi-static representations that directly encode hierarchy changes across multiple steps. Dynamic indented plots visualize changes between successive hierarchies using time-to-space mappings and explicit visual cues for structural updates, allowing multiple timesteps to be compared in a single view [4]. TreeVersity2 supports change exploration through space-filling visualizations and multilevel summaries, enabling users to inspect created, removed, and modified nodes over time [9]. Beyond tree-specific visualizations, Vehlow et al. proposed a matrix-based representation that combines hierarchical grouping, temporal ordering, and flow-based transition cues to support analysis of dynamic hierarchies in evolving graph sequences [21]. While these methods effectively present temporal continuity and preserve structural readability, they are primarily designed to show how hierarchies change across time rather than to explicitly model how one tree transforms into the next. Furthermore, when evolution sequences are long and critical changes are sparse, animation- or snapshot-based views often make it difficult to review distant steps, locate salient transitions, or focus on topology-aware differences while retaining essential structural context.

2.2 Tree Difference computation

The classical tree edit distance formulation introduced by Zhang and Shasha established a foundational model for computing the minimum-cost sequence of insert, delete, and relabel operations required to transform one tree into another [29]. This method provides a principled way to describe structural differences and change processes between two trees. Later work improved the efficiency and robustness of tree edit distance computation. RTED introduced a robust strategy selection mechanism that performs well across different tree shapes [15], while AP-TED further improved efficiency and scalability for practical tree comparison [16]. Approximate or alternative similarity measures, such as pq-Gram distance, have also been proposed to make tree comparison more scalable for large datasets [1].

Beyond generic tree edit distance, several methods address the problem in specific classes of structured data. X-Diff detects changes in XML documents by treating them as unordered trees and generating change descriptions based on structural matching [24]. Earlier work on change detection in hierarchically structured information argued that meaningful change descriptions may require richer structural operations and explicit matching strategies [5]. Tree differencing has also been extensively studied in source code analysis, where abstract syntax trees provide a natural hierarchical representation of programs. ChangeDistiller extracts fine-grained structural edits from ASTs and organizes them into semantically meaningful change types [8]. GumTree further improves AST differencing by combining top-down and bottom-up matching strategies to recover node correspondences and edit scripts that better reflect developer intent, including move operations [7]. More specialized topology-driven differencing methods, such as branch-based edit distances for merge trees, show how

structural comparison can be adapted to topology-centric settings in scientific visualization [3].

Although this literature provides an essential computational foundation, it does not by itself solve the visual analytics problem addressed in our work. Most tree differencing methods are designed to compute distances, correspondences, or edit scripts, rather than to support human-centered analysis of tree evolution. They are therefore less concerned with how differences should be presented to analysts, how unchanged context should be preserved to maintain interpretability, or how local structural edits should be integrated into the analysis of long evolution sequences. These methods are powerful for modeling structural change, but they are not sufficient for topology-aware visual comparison and multi-step evolution analysis.

2.3 Temporal Summarization and Sequence Abstraction

Our work is also related to visual analytics research on temporal sequence summarization. A large body of work in this area studies how to abstract long or complex event sequences into more compact representations that reveal high-level patterns, salient intervals, and representative stages. LifeFlow introduced an overview representation for event sequences that aggregates many temporal records into aligned visual timelines, making it easier to identify common pathways, temporal spacing, and outcome-related patterns [26]. EventFlow extended this direction by providing simplification operations and a visual complexity model, enabling analysts to progressively reduce clutter and discover important temporal structures in large collections of sequences [14]. Outflow uses flow-based visual representations to summarize temporal pathways and transitions across many event sequences [23], while DecisionFlow addresses high-dimensional temporal event data through milestone-based abstractions and interactive multi-view exploration [27].

Subsequent work further explored multilevel abstraction, stage analysis, and progression summarization for complex temporal data. EventThread and ET2 focus on stage analysis, temporal segmentation, and progression understanding, providing methods to identify latent stages and summarize how sequences evolve across them [30, 31]. Sequen-C supports multilevel overviews of temporal event sequences, allowing analysts to move between coarse and fine-grained representations while preserving sequence-level context [25]. Other systems, such as SSRDVis and LoLo, emphasize pattern summarization, anomaly detection, and multilevel exploration for long event sequences with many events [11,28]. This literature is highly relevant to Tree Evolution because long evolution processes often contain only a limited number of critical structural changes, while the rest of the sequence mainly provides contextual continuity.

This literature provides an essential computational foundation for our work. Most tree differencing methods are designed to compute distances, rather than to support human-centered analysis of tree evolution. They are less concerned with how differences should be presented to analysts, how unchanged context should be preserved to maintain interpretability. Building upon these techniques, we propose a configurable tree edit distance method that supports flexible comparison under different application scenarios. Moreover, we leverage the byproducts of edit distance computation to track structural changes, enabling topology-aware visual comparison and multi-step evolution analysis.

2.4 Domain-Specific Analysis of Evolving Trees

Several studies have demonstrated the practical importance of analyzing evolving tree-like structures in specific domains. In database systems, Picasso visualizes optimizer behavior across query parameter spaces and helps users inspect how query plans vary under different conditions [19]. More recently, QOVIS directly supports understanding and diagnosing query optimization by exposing intermediate query plans, transformation logic, and the optimization process through a visualization-assisted framework [20]. These systems are particularly relevant to our work because query plans are a representative example of evolving tree structures.

In software engineering and program analysis, hierarchical program representations also evolve over time. Tree-based version control in Envision explores how AST-aware differencing and merging can support semantically meaningful version management [18]. Related work on dynamic call graphs and call graph evolution further illustrates how tree-like structures can change across program versions or executions, and how visual or analytical methods can help identify recurring patterns, unstable components, or noteworthy structural changes [2, 22]. Tree evolution is also important in search, constraint solving, and scientific data analysis. Visual Search Tree Profiling provides tools for inspecting large search trees, comparing search behaviors, and identifying critical or repeated substructures during search processes [6]. In scientific visualization, Temporal Merge Tree Maps summarizes the evolution of topological features over time by aligning and linearizing merge trees extracted from temporal scalar data [13].

These methods are effective in their target domains, but also limit their generalizability. In contrast, our goal is to develop a general topology-aware visual analytics framework for Tree Evolution that supports corresponding change understanding, context-preserving structural comparison, and long-sequence summarization across different types of tree-structured processes.

3 Task and System Framework

3.1 Analytical task and design requirement

To better understand the analytical needs of Tree Evolution, we consulted researchers from multiple domains in which tree-structured processes naturally arise and evolve over time. These domains include large language model (LLM) reasoning and agent systems, where researchers are interested in how reasoning structures expand, branch, backtrack, or reorganize across inference steps; database query optimization, where query plans are progressively rewritten into final forms; and program analysis and software engineering, where code revisions can be reflected as structural changes in abstract syntax trees (ASTs). Although these domains have different detailed requirement , they share a common need to analyze how tree structures evolve step by step, how local edits affect the overall topology, and how long evolution processes can be transformed into interpretable patterns.

From these discussions, we have identified three common analytical tasks.

T1. Adjacent-tree comparison. A primary need across domains is to compare two consecutive trees as the basic unit of evolution analysis. Analysts want to understand where changes occur in the hierarchy and what kinds of changes they represent, such as node insertion, deletion, attribute update, local restructuring, or subtree replacement. This task is essential because adjacent comparison provides the most direct evidence of the transformation logic between two neighboring states.

T2. Cross-step tracing. Beyond pairwise comparison, analysts need to follow how a node, subtree, or structurally related region evolves across multiple steps. In particular, they want to track whether a node appears, disappears, changes attributes, or shifts its topological position as the process unfolds. Such tracing is important for understanding propagation effects, repeated revisions, rollback behaviors, and the temporal development of critical structures.

T3. Long-sequence simplification. When a tree evolves over many steps, analysts need an overview that reveals the global progression of the process. They are interested in identifying salient intervals, key transition points, repeated editing behaviors, and recurring structural patterns, while still being able to relate these summaries back to detailed local changes. Effective summarization is particularly important when critical edits are sparse relative to the full sequence length.

These analytical tasks suggest that an effective Tree Evolution analysis system should support not only local comparison between adjacent trees, but also the tracing of important structures over time and the summarization of long evolution processes. Accordingly, we derive the following design requirements.

R1. Flexible difference specification. Since different applications may emphasize different forms of change, the system should support flexible specification of what is treated as change and what is treated as invariant, including structural modifications, attribute updates, or both.

R2. Context-preserving difference presentation. The system should automatically detect and present differences between adjacent trees in a way that emphasizes changed regions while preserving sufficient unchanged context for structural interpretation.

R3. Cross-step structure tracing. The system should enable analysts to follow the evolution of nodes or substructures across multiple steps, including their emergence, disappearance, attribute variation, and topological relocation.

R4. Salient transition identification. The system should help analysts identify steps or regions with especially drastic structural changes, providing visual cues that reveal where critical transitions or anomalous edits occur.

R5. Sequence-level evolution summarization. The system should provide compact overviews of long tree evolution sequences, revealing major trends, salient intervals, and recurring structural patterns while maintaining access to detailed local changes.

3.2 System framework

To address the above design requirements, we propose a visual analytics framework for exploring the evolution of temporal tree data, as illustrated in Fig. 2. The framework takes as input a time-ordered sequence of trees $T_1, T_2, T_3, \ldots$, where each tree represents the state of a hierarchical structure at a specific time step. To capture structural evolution, the system first performs adjacent-tree differencing to identify changes between consecutive trees. Specifically, we employ TMED, a tree differencing mechanism that characterizes structural changes in terms of edit operations such as node insertion, deletion, and move/update. Based on the extracted differences, we construct a hierarchical representation termed the minimum-difference forest (MDF), which organizes evolving structures and their changes at multiple levels of granularity. In essence, the MDF serves as an abstraction layer that transforms raw temporal tree sequences into a structured representation suitable for scalable visual analysis.

Built upon the MDF, the framework integrates two coordinated visual components to support the design goals. First, a multi-granularity tree view enables users to inspect structural changes at different levels of detail, thereby supporting both fine-grained change localization (R1) and multi-level exploration from local edits to global patterns (R2). Second, a temporal summary view combines structural change features with temporal information to provide an overview of the evolution process, helping users identify important stages and understand long-term dynamics (R3). By organizing changes hierarchically and presenting them through coordinated views, the framework also reduces visual clutter and improves the scalability of temporal tree analysis (R4). The following sections detail the TMED-based differencing process, the construction of the MDF, and the corresponding visual design and interaction techniques.

4 Data Model and Problem Definition

4.1 Tree, Tree Sequence, and Evolution

We consider a rooted tree as the basic structural unit in our data. A tree is defined as

$$T = (V, E, r),$$

where V is the set of nodes, $E \subseteq V \times V$ is the set of directed parent-child edges, and $r \in V$ is the root node. Each node $v \in V$ may be associated with a label, an attribute vector, or other application-specific metadata. In the following, we mainly focus on the hierarchical structure of the tree, while allowing node attributes to be incorporated when needed.

For a node $v \in V$, we denote by $parent(v)$ its parent node, and by $children(v)$ the set of its child nodes. The subtree rooted at v is denoted by

$$T(v) = (V(v), E(v), v),$$

where $V(v) \subseteq V$ contains v and all its descendants, and $E(v) \subseteq E$ contains the edges induced by $V(v)$.

To model structural evolution over time, we consider not a single tree, but an ordered sequence of trees:

$$\mathscr{T} = \langle T_1, T_2, \ldots, T_n \rangle,$$

where each T_i represents the tree observed at the i-th time step, version, or evolutionary stage. We refer to $\mathscr{T}$ as a *tree sequence*. The ordering of trees may be induced by time, iteration, version number, or any domain-specific progression. In this work, we assume that the sequence order is meaningful and that structural evolution can be analyzed by comparing adjacent trees in the sequence.

Given a tree sequence $\mathscr{T}$, we define *tree evolution* as the structural transformation process across consecutive trees. Rather than modeling evolution only at the level of the full sequence, we decompose it into a series of local transitions between adjacent trees. For two adjacent trees T_i and T_{i+1}, the transition

$$\delta_i : T_i \to T_{i+1}$$

is referred to as an *evolution step*. Each evolution step captures the structural changes occurring between two consecutive states of the tree sequence.

This adjacency-based formulation has two advantages. First, it enables analysts to inspect evolution incrementally, which is often more cognitively manageable than comparing distant trees directly. Second, it provides a natural basis for identifying local structural changes and tracking how such changes accumulate over the entire sequence. In this work, our primary analytical focus is on the comparison of adjacent trees, i.e., the characterization of each evolution step δ_i.

4.2 Structural Difference and Difference Nodes

To quantify the amount of change in an evolution step, we model the difference between T_i and T_{i+1} using tree edit operations. Specifically, we consider a transformation sequence that converts T_i into T_{i+1} through a set of elementary edit operations, such as node insertion, node deletion, and node relabeling or substitution, depending on the application setting.

The *tree edit distance* between two adjacent trees is defined as the minimum cost required to transform one tree into the other:

$$\mathrm{TED}(T_i, T_{i+1}).$$

This value provides a quantitative measure of structural difference. A larger edit distance indicates a greater amount of structural change between the two adjacent stages.

However, the edit distance alone only reflects *how much* the structure has changed, but does not explicitly reveal *where* the changes occur or how they are structurally related. Therefore, beyond this scalar quantity, we derive a set of structural difference elements from the edit sequence.

Let

$$D_i = \{d_1, d_2, \ldots, d_m\}$$

denote the set of nodes involved in the minimum edit sequence between T_i and T_{i+1}. We refer to these nodes as *difference nodes*. Intuitively, a difference node marks a local position in the tree where a structural modification occurs in the corresponding evolution step. Difference nodes constitute the finest-grained representation of structural change. They preserve the exact locations of local edits and therefore provide maximal detail about the transformation between adjacent trees.

Nevertheless, directly presenting all difference nodes is often insufficient for effective analysis. When the number of changes is large, or when changed nodes are scattered across the hierarchy, a pure node-level representation may impose excessive cognitive burden and obscure higher-level structural patterns. This motivates the need for more structured abstractions built on top of difference nodes.

4.3 Difference Subtrees and Difference Forests

To organize difference nodes into interpretable structural units, we introduce the concepts of *difference subtree* and *difference forest*.

For a tree T and a node $r \in V$, the subtree $T(r)$ is called a difference subtree if it contains at least one difference node. More generally, for a subset of difference nodes $X \subseteq D_i$, if all nodes in X are contained in $T(r)$, then $T(r)$ is said to be a difference subtree for X. Among all such

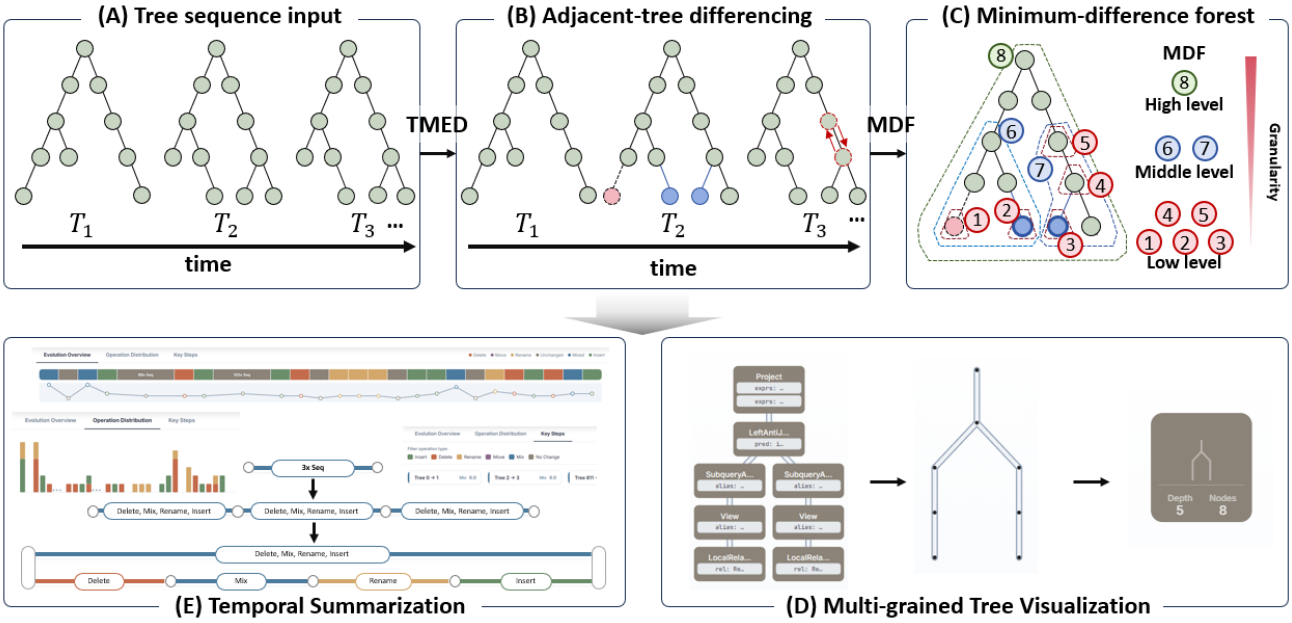

Fig. 2: The workflow of TRACE. Need description

subtrees that cover X, we define the one with the smallest structural extent as the *minimum difference subtree* for X, denoted as

$$T^*(X).$$

Depending on the precise implementation, the structural extent may be measured by the number of nodes, the number of edges, or the induced hierarchical span. In the simplest setting, $T^*(X)$ can be understood as the smallest rooted subtree that covers all nodes in X.

The difference subtree provides a localized structural context for a set of changes. Rather than showing isolated difference nodes one by one, it groups nearby or structurally related differences into a higher-level region, thereby supporting more interpretable qualitative analysis.

In many evolution steps, difference nodes are distributed across multiple disconnected or weakly related regions of the tree. In such cases, a single difference subtree may be too coarse to summarize the changes effectively. We therefore define a *difference forest* as a set of pairwise disjoint difference subtrees that jointly cover a target set of difference nodes. Formally, let $X \subseteq D_i$ be a set of difference nodes. A set of subtrees

$$F = \{T(r_1), T(r_2), \ldots, T(r_k)\}$$

is called a difference forest for X if the following two conditions hold:

$$X \subseteq \bigcup_{j=1}^{k} V(r_j),$$

and

$$V(r_p) \cap V(r_q) = \emptyset, \quad \forall p \neq q.$$

That is, every node in X is covered by at least one subtree in the forest, and any two subtrees in the forest are non-overlapping.

A *minimum difference forest* for X is a difference forest in which each subtree is a minimum difference subtree for the subset of difference nodes it covers. Importantly, for a given set of difference nodes, multiple valid minimum difference forests may exist. This non-uniqueness reflects the fact that structural changes can be summarized at multiple granularities, depending on how one chooses to partition the difference nodes into disjoint structural regions.

Difference forests serve as an intermediate representation between two extremes:

- presenting every difference node individually, which preserves maximal detail but may lead to clutter; and
- summarizing all changes with a single large subtree, which reduces complexity but may obscure local distinctions.

This representation is particularly suitable for visual analysis, as it enables structured summaries of differences at multiple levels of abstraction while preserving a clear, non-overlapping decomposition of changed regions.

4.4 Difference Summarization and Problem Statement

The definitions above establish a hierarchy of difference representations:

- *tree edit distance* captures the overall magnitude of change;
- *difference nodes* capture exact local changes;
- *difference subtrees* capture local structural contexts of change; and
- *difference forests* capture structured summaries of distributed changes.

Based on this hierarchy, we formulate adjacent-tree comparison as a *difference summarization problem*. Given two adjacent trees T_i and T_{i+1}, we aim to derive a representation that is both informative and cognitively manageable.

A trivial solution is to directly show all difference nodes. This preserves all local details but may overwhelm analysts when changes are numerous. Another trivial solution is to aggregate all difference nodes into a single large subtree. This reduces the number of visual elements but may hide important structural details. Therefore, the central challenge is to identify an appropriate difference forest that balances *detail preservation* and *representation simplicity*. This trade-off forms the basis of our method.

Given a tree sequence

$$\mathscr{T} = \langle T_1, T_2, \ldots, T_n \rangle,$$

our task is to analyze each evolution step $\delta_i : T_i \to T_{i+1}$ by jointly supporting:

- *quantitative comparison*, through the tree edit distance $\mathrm{TED}(T_i, T_{i+1})$; and
- *qualitative comparison*, through a structured summary of the corresponding difference nodes.

More specifically, for each adjacent tree pair (T_i, T_{i+1}), we seek to construct a difference forest F_i over the difference node set D_i such that:

- all important structural differences are covered;
- the resulting summary remains structurally interpretable; and
- the complexity of inspecting the changes is reduced without sacrificing excessive detail.

In the next section, we describe how this data model supports the design of our method for constructing and selecting appropriate difference forests.

4.5 Difference Forest Construction Algorithm

Based on the above data model, we formulate difference forest construction as a constrained grouping problem over the set of difference nodes. Given the difference node set D_i, our goal is to construct a difference forest

$$F_i = \{T(r_1), T(r_2), \ldots, T(r_k)\}$$

such that all difference nodes are covered, the subtrees are pairwise disjoint, and the resulting forest achieves a balance between preserving local details and reducing cognitive complexity.

Basic idea. The key idea is to treat each difference node as an initial atomic group and then iteratively merge structurally related groups in a bottom-up manner. Each group of difference nodes is represented by its minimum difference subtree. The algorithm starts from the finest-grained representation, where each difference node forms an individual subtree, and progressively combines groups if the merge leads to a better trade-off between structural fidelity and representational simplicity. The merging process terminates when no valid merge can further improve the objective.

Group representation. Let $G = \{g_1, g_2, \ldots, g_k\}$ denote a partition of the difference node set D_i, where each group $g_j \subseteq D_i$ contains one or more difference nodes, and

$$\bigcup_j g_j = D_i, \qquad g_p \cap g_q = \emptyset \text{ for } p \neq q.$$

For each group g_j, we compute its minimum difference subtree $T^*(g_j)$. The corresponding difference forest is therefore

$$F_i(G) = \{T^*(g_1), T^*(g_2), \ldots, T^*(g_k)\}.$$

A valid forest additionally requires that these subtrees are pairwise disjoint.

Objective function. To evaluate the quality of a difference forest, we define an objective function that balances three criteria: contextual redundancy, intra-subtree dispersion, and forest complexity. Specifically, for a forest F, we define

$$\mathscr{L}(F) = \alpha \cdot C_{\text{extra}}(F) + \beta \cdot C_{\text{disp}}(F) + \lambda \cdot C_{\text{count}}(F),$$

where α, β, and λ are user-controllable weights.

The first term measures the amount of redundant context introduced by the forest:

$$C_{\text{extra}}(F) = \sum_{S \in F} \left(|V(S)| - |D_i \cap V(S)| \right).$$

This term penalizes subtrees that include many non-difference nodes and therefore become overly coarse.

The second term measures how dispersed the difference nodes are within each subtree:

$$C_{\text{disp}}(F) = \sum_{S \in F} \frac{1}{|D_S|} \sum_{d \in D_S} dist(root(S), d),$$

where $D_S = D_i \cap V(S)$ and $dist(root(S), d)$ denotes the tree distance from the root of subtree S to difference node d. This term favors subtrees whose covered difference nodes are structurally compact.

The third term controls the complexity of the forest:

$$C_{\text{count}}(F) = |F|.$$

This term penalizes forests with too many subtrees and helps reduce visual and cognitive fragmentation.

Our goal is to find a forest that minimizes $\mathscr{L}(F)$ subject to the coverage and non-overlap constraints:

$$D_i \subseteq \bigcup_{S \in F} V(S),$$

$$V(S_p) \cap V(S_q) = \emptyset, \quad \forall S_p, S_q \in F,\ p \neq q.$$

Greedy bottom-up optimization. We adopt a greedy bottom-up optimization strategy. Initially, each difference node $d \in D_i$ forms a singleton group $g = \{d\}$, and the initial forest consists of the corresponding minimum difference subtrees. At each iteration, the algorithm considers merging two existing groups g_a and g_b into a new group

$$g_{ab} = g_a \cup g_b.$$

The merged group is represented by the minimum difference subtree $T^*(g_{ab})$.

A merge is considered valid only if the new subtree remains disjoint from all other subtrees currently in the forest. For each valid candidate merge, the algorithm computes the updated forest and evaluates the change in objective value:

$$\Delta\mathscr{L} = \mathscr{L}(F') - \mathscr{L}(F).$$

The algorithm then selects the merge with the largest reduction in objective value. If the best candidate satisfies $\Delta\mathscr{L} < 0$, the merge is accepted and the forest is updated. Otherwise, the process terminates.

This strategy naturally proceeds from fine-grained local differences to coarser structural summaries, and the optimization stops at the point where further abstraction would introduce more loss than benefit.

Algorithm. The procedure is summarized below.

```
Algorithm 1: Difference Forest Construction
Input: rooted tree T, difference node set D_i, weights alpha, bet
Output: pairwise disjoint difference forest F

Initialize groups G = { {d} | d in D_i }
Compute F = { T*(g) | g in G }
repeat
    bestDelta <- 0
    bestMerge <- null
    for each pair (g_a, g_b) in G do
        g_ab <- g_a union g_b
        S_ab <- T*(g_ab)
        if S_ab overlaps any subtree in F \ {T*(g_a), T*(g_b)}
            continue
        end if
        F' <- (F \ {T*(g_a), T*(g_b)}) union {S_ab}
        delta <- L(F') - L(F)
        if delta < bestDelta then
            bestDelta <- delta
            bestMerge <- (g_a, g_b, S_ab)
        end if
    end for
    if bestMerge != null then
        update G and F using bestMerge
    end if
until bestMerge == null
return F
```

Discussion. The proposed algorithm has several desirable properties. First, it is consistent with the hierarchical structure of trees: each merge corresponds to a structurally meaningful aggregation of difference nodes. Second, because disjointness is explicitly enforced during

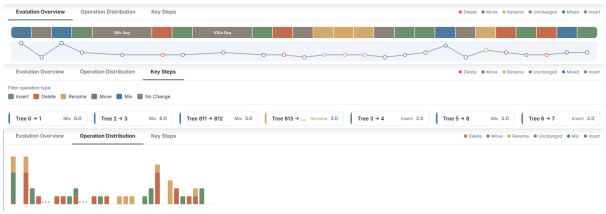

Fig. 3: Temporal view

optimization, the resulting forest always satisfies the non-overlap constraint in our definition. Third, the algorithm can naturally produce multi-granularity results: the intermediate forests generated during optimization form a hierarchy of summaries, allowing analysts to move from fine-grained local changes to coarser structural overviews.

5 Visualization

As illustrated in Fig. 2, the visual design of TRACE follows Shneiderman's mantra of *overview first, zoom and filter, then details on demand*. To support the analysis of temporal tree evolution at multiple levels of abstraction, the interface is organized into three coordinated components: a *statistics view* for global temporal overview, a *hierarchy view* for intermediate-level exploration within a selected time range, and a set of *tree views* for fine-grained structural inspection. Together, these components enable analysts to move progressively from global temporal patterns to local structural details while preserving contextual awareness across different stages of analysis.

The *statistics view* is positioned at the top of the interface and provides a global summary of time-oriented statistics over the entire tree sequence. It helps analysts understand overall evolution trends, identify salient transition steps, and locate time intervals with substantial structural variation. Based on this overview, analysts can select a time range of interest for further exploration. Once a temporal interval is selected, the *hierarchy view* appears below the statistics view and serves as an overview of the selected region. It presents the corresponding evolution sequence in a compact hierarchical manner and supports multi-level inspection of structural changes within the chosen interval. Beneath the hierarchy view, a set of *tree views* preserves the finest-grained tree structures and exposes node-level edits for detailed inspection. In this way, the three coordinated views form a progressive exploration pipeline that bridges global temporal context, intermediate structural abstraction, and detailed tree-level analysis.

5.1 Statistics View

The statistics view provides the global entry point for temporal analysis, shown as Fig. 3. Its main purpose is to summarize how structural change is distributed along the evolution sequence, allowing analysts to first understand the overall process before inspecting individual trees or local edits. Instead of presenting every evolution step with equal emphasis, the statistics view highlights the relative intensity, frequency, or concentration of structural changes over time, thereby helping users quickly identify salient intervals and potential transition points.

To support this goal, the statistics view encodes time-oriented summary measures derived from adjacent-tree differencing, such as change magnitude, the number of difference nodes, or the distribution of difference-aware structures across steps. These summaries give analysts a compact overview of whether the sequence evolves smoothly, exhibits abrupt transitions, or contains repeated bursts of activity. The view therefore directly supports sequence-level summarization and salient transition identification by revealing where analysts should focus their attention in long evolution sequences.

The statistics view also serves as the primary mechanism for temporal selection. Analysts can choose a contiguous time interval based on the observed summaries and pass the selected region to the downstream views. This interaction follows the *zoom and filter* stage of the visual analysis process: the statistics view first presents the global context, and

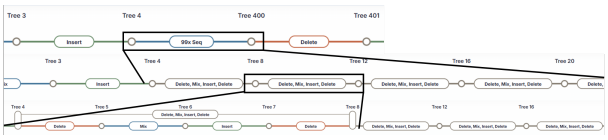

Fig. 4: Temporal view

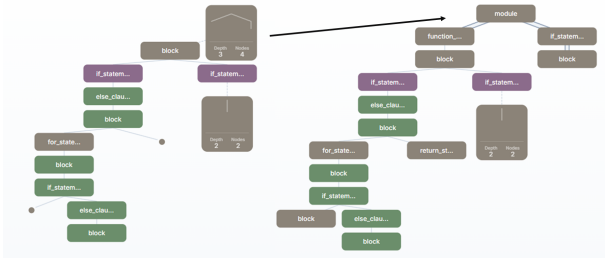

Fig. 5: Tree view

then guides analysts toward a subset of steps that merit more detailed structural investigation.

5.2 Hierarchy View

After a time interval is selected in the statistics view, the hierarchy view, shown as Fig. 4, provides an intermediate-level overview of the corresponding evolution region. Its role is not to expose all node-level details immediately, but to organize the selected steps into a structured representation that preserves the major hierarchical relationships among changes. In this sense, the hierarchy view acts as the bridge between global temporal summarization and detailed structural inspection.

The hierarchy view visualizes the evolution sequence within the selected interval in a compact form and supports multi-granularity analysis of changed regions. Built upon the MDF representation, it allows analysts to examine how local edits aggregate into larger structural units and how these units evolve across multiple steps. This intermediate abstraction is particularly important when changes are scattered across different parts of the tree: it helps analysts reason about whether these edits belong to the same evolving branch, correspond to repeated revisions, or indicate multiple independent transformation regions.

By presenting change structures at multiple levels of granularity, the hierarchy view supports context-preserving exploration. Analysts can inspect the selected interval at a coarse level to understand the overall structural organization of the changes, and then progressively refine their focus toward smaller substructures of interest. This view therefore supports both cross-step tracing and localized exploration while avoiding the clutter that would arise from showing all detailed trees at once.

5.3 Tree Views

The tree views (shown as Fig. 5) occupy the bottom portion of the interface and provide details on demand. While the statistics view summarizes temporal patterns and the hierarchy view organizes changes at an intermediate level, the tree views preserve the finest-grained structural information of the selected trees. They allow analysts to inspect the original tree topology, identify exact node-level edits, and verify hypotheses formed from the higher-level views.

Each tree view presents a detailed tree structure together with the local differences associated with a selected step or structural region. Through this design, analysts can examine concrete edit operations such as insertion, deletion, or move/update, and understand how these operations are embedded within the surrounding unchanged topology. The preservation of detailed context is essential because isolated change markers are often insufficient for interpretation; analysts need to see both the edit itself and its structural neighborhood.

The tree views are coordinated with the hierarchy view and the statistics view. Selections made at higher levels narrow down which trees

and which structural regions are displayed in detail, while observations from detailed inspection can be traced back to their broader temporal and hierarchical contexts. In addition, we draw linking lines across adjacent trees: when analysts hover over a node, unchanged corresponding nodes in the neighboring trees are connected, allowing users to visually trace how a node is simplified or preserved across steps. This coordination allows analysts to move flexibly between overview and detail without losing orientation, thereby supporting efficient exploration of both local edits and long-range evolutionary patterns.

5.4 Interactions

The three views are linked through a set of coordinated interactions that support progressive exploration. Analysts begin with the statistics view to identify salient intervals in the temporal sequence, then select a time range for focused analysis. The hierarchy view summarizes the selected interval and allows users to inspect changed regions at different levels of granularity. Finally, the tree views reveal the detailed structures associated with selected steps or subregions.

This interaction design supports a natural analysis workflow: analysts first obtain a global overview of temporal dynamics, then filter the sequence to a region of interest, and finally inspect detailed structures on demand. Because selections are propagated across views, the interface maintains context during transitions between levels of abstraction. As a result, analysts can connect individual edits to larger structural units and relate these units back to sequence-level temporal patterns.

6 Case Study

We demonstrate the effectiveness of TRACE through three case studies across different domains: database query optimization, code evolution, and agent memory management. These cases were selected because they exhibit distinct tree evolution characteristics and analytical goals, allowing us to examine how TRACE supports temporal overview, multi-granularity structural analysis, and detailed change inspection in practice.

6.1 Case Study 1: Detecting Loop Optimization in Database Query Optimizer

A database query optimizer transforms an input SQL query into an efficient execution plan through a sequence of rewrite and optimization operations. In practice, however, interactions among optimization rules may lead to repeated rewrites or even optimization loops, where similar transformations are applied multiple times. Such behaviors are difficult to detect using existing tools, especially when analysts need to manually inspect long optimization traces and compare repeated plan transformations.

TRACE helps analysts identify these repeated optimization patterns without requiring strong domain-specific prior knowledge. In the statistics view, the Evolution Overview compresses recurring operation subsequences and exposes loop-like patterns as repeated summarized units, allowing analysts to quickly detect suspicious repetitive behaviors in long optimizer traces. The difference trend line and operation distribution further reveal intervals with repeated bursts of structural activity, helping users localize the suspicious region. After selecting the relevant time interval, the hierarchy view summarizes the repeated plan transformations at multiple levels of granularity, while the tree views expose the detailed node-level edits associated with each iteration. Through this workflow, analysts can recognize that the optimizer repeatedly applies similar rewrites to structurally related subtrees, thereby revealing inefficient or redundant optimization behavior.

6.2 Case Study 2: Revealing Code Refactoring Patterns through AST Evolution

Abstract syntax trees (ASTs) provide a hierarchical representation of program structure. During software development, code revisions naturally induce AST evolution, reflecting operations such as function insertion, variable renaming, statement reorganization, and subtree deletion. In our case, each AST contains roughly 100–200 nodes corresponding to Python syntax elements such as `Module`, `FunctionDef`, `Assign`, `If`, `For`, `Return`, and `Call`. The resulting evolution sequence includes structural modifications such as inserting new functions, renaming variables or functions, moving code blocks, and removing outdated statements.

This case highlights the value of TRACE for understanding refactoring patterns in large and partially stable trees. Since most of the AST remains unchanged across adjacent revisions, direct comparison can easily be overwhelmed by unchanged context. TRACE addresses this issue by using difference-aware highlighting and multi-granularity summarization. In the tree views, unchanged nodes are visually simplified while changed nodes are emphasized through visual encoding, making local edits easier to recognize. The glyph-based representation of minimum difference forests reduces cognitive load while preserving sufficient structural context for interpretation. Moreover, when analysts hover over a node, unchanged corresponding nodes in neighboring trees are linked by lines, making it possible to visually trace how a node or subtree is preserved, simplified, or reorganized across revisions. This interaction helps reveal refactoring patterns such as repeated local restructuring, systematic renaming, and incremental function decomposition.

6.3 Case Study 3: Understanding Memory Organization in Agent Memory Trees

Large language model (LLM) agents often maintain evolving memory structures to store observations, reflections, and higher-level summaries of interaction histories. As the dialogue continues, the memory tree grows and changes through insertion, aggregation, rewriting, and occasional deletion. A key analytical problem is to understand how low-level observations are gradually organized into higher-level reflections and how the memory structure evolves over long interaction sessions.

In this case, we analyze the memory tree of an LLM agent over more than 15 dialogue rounds. The memory structure follows a hierarchical organization similar to prior generative agent frameworks, with high-level categories such as agent persona, reflections, and observations. The dominant evolution pattern is insertion, corresponding to the agent continuously incorporating new information, together with occasional renaming or restructuring when existing memories are refined or reorganized.

TRACE helps reveal both the global trend and the local mechanisms of this memory evolution process. In the statistics view, the overview highlights long insertion-dominated intervals while also exposing a few salient transition points with substantial restructuring. The hierarchy view reveals how many low-level observation nodes are progressively grouped into newly formed reflection nodes, making the memory consolidation process visible at an intermediate structural level. In the tree views, analysts can further inspect how multiple observations are absorbed into a new reflection subtree and how the surrounding memory structure is updated accordingly. Through these coordinated views, TRACE makes the agent's memory organization process interpretable, showing not only that the tree grows over time, but also how the agent gradually transforms raw observations into more abstract memory structures.

6.4 Summary

Across the three cases, TRACE supports analysts in identifying salient intervals, detecting repeated or abnormal structural behaviors, and tracing how local changes accumulate into larger evolutionary patterns. The database query optimization case demonstrates the ability of TRACE to reveal repeated transformation loops, the AST case shows its value for exposing refactoring patterns in partially stable trees, and the agent memory case highlights its usefulness for understanding hierarchical memory organization over time. Together, these studies suggest that TRACE can effectively support tree evolution analysis in diverse domains with different semantic meanings but similar structural dynamics.

References

[1] N. Augsten, M. Böhlen, and J. Gamper. The pq-gram distance between ordered labeled trees. *Journal of Discrete Algorithms*, 2010. 2

[2] M. Bao et al. Call graph evolution analytics over a version series of an evolving software system. In *ASE*, 2022. 3

[3] C. Bauer et al. Branch decomposition-independent edit distances for merge trees. *Computer Graphics Forum*, 2022. 3

[4] M. Burch et al. Visualizing hierarchy changes by dynamic indented plots. In *IEEE VIS*, 2014. 2

[5] S. S. Chawathe, A. Rajaraman, H. Garcia-Molina, and J. Widom. Change detection in hierarchically structured information. In *SIGMOD*, 1996. 2

[6] J. Dubois et al. Visual search tree profiling. *Constraints*, 2015. 3

[7] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus. Fine-grained and accurate source code differencing. In *ASE*, 2014. 2

[8] B. Fluri, M. Wursch, M. Pinzger, and H. C. Gall. Change distilling: Tree differencing for fine-grained source code change extraction. *IEEE Transactions on Software Engineering*, 2007. 2

[9] J. Guerra-Gómez et al. TreeVersity2 – visualizing change over time using dynamic hierarchies. In *EuroVis*, 2013. 2

[10] D. Hahn et al. Visualization of varying hierarchies by stable layout of voronoi treemaps. In *EuroVis*, 2014. 2

[11] J. H. Isenberg et al. LoLo: A visual analytics approach for long sequences with a lot of events. *Information Visualization*, 2025. 3

[12] M. Köpp and T. Weinkauf. Temporal treemaps: Static visualization of evolving trees. *Computer Graphics Forum*, 2018. EuroVis 2018. 2

[13] M. Köpp and T. Weinkauf. Temporal merge tree maps: A topology-based static visualization for temporal scalar data. *IEEE Transactions on Visualization and Computer Graphics*, 2022. 3

[14] M. Monroe, R. Lan, J. Delfi, G. Ellis, C. Plaisant, and B. Shneiderman. Temporal event sequence simplification. In *CHI*, 2013. EventFlow. 3

[15] M. Pawlik and N. Augsten. RTED: A robust algorithm for the tree edit distance. *Information Systems*, 2011. 2

[16] M. Pawlik and N. Augsten. Efficient computation of the tree edit distance. *ACM Transactions on Database Systems*, 2015. AP-TED. 2

[17] L. Scheibel et al. EvoCells: A treemap layout algorithm for evolving tree data. In *International Joint Conference on Computer Vision, Imaging and Computer Graphics*, 2018. 2

[18] W. S. Schneider. Tree-based version control in envision. Master's thesis, Unknown Institution, 2015. 3

[19] S. Sudarshan et al. Picasso: A database query optimizer visualizer. *Proceedings of the VLDB Endowment*, 2010. 3

[20] H. Tian et al. QOVIS: Understanding and diagnosing query optimizer via a visualization-assisted approach. *Proceedings of the VLDB Endowment*, 2025. 3

[21] C. Vehlow, F. Beck, and D. Weiskopf. Visualizing dynamic hierarchies in graph sequences. *IEEE Transactions on Visualization and Computer Graphics*, 2016. 2

[22] E. von Praun et al. Visualizing dynamic call graphs. In *VMV*, 2012. 3

[23] F. Wang et al. Outflow: Visualizing patient flow by flow map layouts. *IEEE Transactions on Visualization and Computer Graphics*, 2014. 3

[24] Y. Wang, D. J. DeWitt, and J.-Y. Cai. X-Diff: An effective change detection algorithm for XML documents. In *ICDE*, 2003. 2

[25] Y. Wang et al. Sequen-C: A multilevel overview of temporal event sequences. *IEEE Transactions on Visualization and Computer Graphics*, 2022. 3

[26] K. Wongsuphasawat, J. A. Guerra Gómez, C. Plaisant, T. Wang, M. Taieb-Maimon, and B. Shneiderman. LifeFlow: Visualizing an overview of event sequences. In *CHI*, 2011. 3

[27] P. Xu et al. DecisionFlow: Visual analytics for high-dimensional temporal event sequence data. *IEEE Transactions on Visualization and Computer Graphics*, 2014. 3

[28] L. Yang et al. SSRDVis: Interactive visualization for sequence summarization and rare behavior detection. *Journal of Visualization*, 2019. 3

[29] K. Zhang and D. Shasha. Simple fast algorithms for the editing distance between trees and related problems. *Algorithmica*, 1989. 2

[30] J. Zhao et al. EventThread: Visual summarization and stage analysis of event sequence data. *IEEE Transactions on Visualization and Computer Graphics*, 2018. 3

[31] J. Zhao et al. Visual progression analysis of event sequence data. *Computer Graphics Forum*, 2018. ET2. 3

A About Appendices

Refer to **??** for instructions regarding appendices.

B Troubleshooting

B.1 ifpdf error

If you receive compilation errors along the lines of `Package ifpdf Error: Name clash, \ifpdf is already defined` then please add a new line `\let\ifpdf\relax` right after the `\documentclass[journal]{vgtc}` call. Note that your error is due to packages you use that define `\ifpdf` which is obsolete (the result is that `\ifpdf` is defined twice); these packages should be changed to use `ifpdf` package instead.

B.2 pdfendlink error

Occasionally (for some LaTeX distributions) this hyper-linked bibTeX style may lead to **compilation errors** (`pdfendlink ended up in different nesting level ...`) if a reference entry is broken across two pages (due to a bug in `hyperref`). In this case, make sure you have the latest version of the `hyperref` package (i.e. update your LaTeX installation/packages) or, alternatively, revert back to `\bibliographystyle{abbrv-doi}` (at the expense of removing hyperlinks from the bibliography) and try `\bibliographystyle{abbrv-doi-hyperref}` again after some more editing.